

# Deep Learning and Causal AI for Estimation

## Lecture 2: Large Language Models

Stephen Hansen  
University College London



FUNDACIÓN  
RAMÓN ARECES



Center for  
International  
Finance

# Introduction

Modern AI systems are largely built on top of Large Language Models.

The first task an LLM is trained on is word prediction → result is a **foundation** model.

A generative/autoregressive model predicts next word in a sequence; a bidirectional model predicts missing words in a sequence.

Fitting a model for word prediction is sometimes called **pre-training** and is wholly self-supervised.

Word prediction is necessary but not sufficient for an LLM to produce high-quality output. Aligning the model to user needs requires **post-training** with human input (see next lecture).

# Word Prediction and Deep Learning

The computer science literature studied sequence prediction for many years with limited success at scale.

Key breakthrough came from **Transformer** neural network architecture which builds on attention mechanism [Vaswani et al., 2017].

LLMs carry on a longer tradition of deep learning models achieving stunning predictive success with black-box structure.

Same forces that make deep learning effective at other problems also apply to LLMs (data, compute, training algorithms).

# Why are LLMs Special?

From the early 2010s, it was clear that deep learning could solve long-standing prediction problems in various domains.

Why is word prediction seen as fundamentally distinct from other prediction tasks?

One answer is that the range of problems that can be formulated in natural language is vastly greater than in other domains.

As such, LLMs are seen as a path to so-called Artificial General Intelligence (whose definition remains murky).

Active debate about whether LLMs are the right framework for achieving AGI and whether they recover a correct 'world model'.

## Pope Leo View

So-called artificial intelligences do not undergo experiences, do not possess a body, do not feel joy or pain, do not mature through relationships and do not know from within what love, work, friendship or responsibility mean. Nor do they have a moral conscience, since they do not judge good and evil, grasp the ultimate meaning of situations or bear responsibility for consequences. They may imitate language, behavior and analytical skills, or even simulate empathy and understanding, but they do not understand what they produce, for they lack the affective, relational and spiritual perspective through which human beings grow in wisdom.

# Outline

Basic Concepts

Transformer Models

Using Generative Model for Measurement

Bidirectional Models

# Tokenization

There is a corpus composed of  $D$  documents indexed by  $d$ .

Raw strings must be converted into sequences of **tokens** which form the linguistic units the model learns to represent.

After tokenization, each document is a length- $N_d$  sequence of tokens  $\mathbf{w}_d = (w_{d,1}, \dots, w_{d,N_d})$ .

The  $V$  **unique** tokens in the corpus are called the **vocabulary** and each has an index  $v$ .

We can then map each term in the corpus into this index, so that  $w_{d,n} \in \{1, \dots, V\}$ .

**Important:** each LLM comes with a custom tokenizer that must be applied before passing text into the model.

# Example

Consider three documents:

1. 'stephen is nice'
2. 'john is also nice'
3. 'george is mean'

We can consider the set of unique terms as {stephen, is, nice, john, also, george, mean} so that  $V = 7$ .

Construct the following index:

| stephen | is | nice | john | also | george | mean |
|---------|----|------|------|------|--------|------|
| 1       | 2  | 3    | 4    | 5    | 6      | 7    |

We then have  $\mathbf{w}_1 = (1, 2, 3)$ ;  $\mathbf{w}_2 = (4, 2, 5, 3)$ ;  $\mathbf{w}_3 = (6, 2, 7)$ .

# What is a Token?

In traditional text analysis, tokenization involves removing common/rare words and consolidating remaining words via stemming/lemmatizing.

Resulting tokens are units of linguistic meaning, and the process simplifies the original text.

# What is a Token?

In traditional text analysis, tokenization involves removing common/rare words and consolidating remaining words via stemming/lemmatizing.

Resulting tokens are units of linguistic meaning, and the process simplifies the original text.

In LLMs, tokenization retains the original structure of the input text without removing information.

Individual tokens are no longer exclusively units of linguistic meaning but include subwords that can be composed to represent words outside the vocabulary.

Allows LLMs to encode words not seen in training data, handle typos, operate in multilingual settings.

# Language Model

A **language model** is a probability distribution over sequences.

By the chain rule this decomposes as

$$p(w_1, \dots, w_N) = \prod_{n=1}^N p(w_n \mid \underbrace{w_1, \dots, w_{n-1}}_{\mathbf{w}_{<n}})$$

Next-token prediction characterizes the full joint distribution.

LLMs use the approximation that

$$p(w_n \mid w_1, \dots, w_{n-1}) \approx p(w_n \mid w_{\max\{n-C, 1\}}, \dots, w_{n-1})$$

where  $C$  is the **context window**, now often exceeding 100,000.

Beyond context truncation, no Markov structure imposed.

# Outline

Basic Concepts

Transformer Models

Using Generative Model for Measurement

Bidirectional Models

# Sequence Representation

Transformers use a feedforward neural network to form a  $V$ -dimensional distribution over the token assignment for  $w_{n+1}$  given  $w_1, \dots, w_n$ .

But how to represent the sequence? Fundamental problem is the dimensionality of the feature space.

Key idea is to *embed* the sequence in a low-dimensional vector space.

Representation starts by endowing each unique token  $v$  with its own embedding vector  $\rho_v \in \mathbb{R}^K$ .

In early neural language models [Mikolov et al., 2013] word embeddings were an object of interest in their own right.

Transformers instead allow word embeddings to interact to give an embedded representation to a whole sequence.

# Example of Next-Word Prediction Problem

*After a season of positive corporate earnings announcements driven by AI adoption, NVIDIA's share price hit an all-time [MASK].*

Which words are most likely to underlie [MASK]?

## Two Alternative Examples with Six Words Removed

*After a season of ~~positive~~ corporate earnings announcements driven by AI adoption, ~~NVIDIA's~~ share price hit an all-time [MASK].*

*After a season of positive corporate earnings announcements driven by AI adoption, NVIDIA's share price hit an all-time [MASK].*

# Attention Weights

The basic idea of attention [Vaswani et al., 2017] is to define a weight  $\alpha_{n,m}$  for each **attended** word  $n$  and **context** word  $m$ .

Normalized in autoregressive language models so that  $\sum_{m=1}^n \alpha_{n,m} = 1$ .

Attention weights highlight the relevant parts of the context preceding each word.

Weights are estimated during neural network training to optimize the quality of word prediction tasks.

# Parameterization of Attention

Let  $\mathbf{W}_q \in \mathbb{R}^{R \times K}$  and  $\mathbf{W}_k \in \mathbb{R}^{R \times K}$  be **query** and **key** weight matrices.

Steps to generate attention weight  $\alpha_{n,m}$ :

1. Form query vector  $\mathbf{q}_n = \mathbf{W}_q \boldsymbol{\rho}_n$
2. Form key vector  $\mathbf{k}_m = \mathbf{W}_k \boldsymbol{\rho}_m$
3. Compute score  $\tilde{\alpha}_{n,m} = \frac{\mathbf{q}_n \cdot \mathbf{k}_m}{\sqrt{R}}$
4.  $\alpha_{n,m} = \frac{\exp(\tilde{\alpha}_{n,m})}{\sum_{m'=1}^n \exp(\tilde{\alpha}_{n,m'})}$

# Using Attention to Update Embeddings

We have an initial embedding  $\rho_n^{(i)}$  for the  $n$ th element in the sequence.

We obtain a new representation as

$$\rho_n^{(i+1)} = \rho_n^{(i)} + \sum_{m=1}^n \alpha_{n,m} \rho_m^{(i)}$$

# Using Attention to Update Embeddings

We have an initial embedding  $\rho_n^{(i)}$  for the  $n$ th element in the sequence.

We obtain a new representation as

$$\rho_n^{(i+1)} = \rho_n^{(i)} + \sum_{m=1}^n \alpha_{n,m} \rho_m^{(i)}$$

Additional architectural details for full Transformer block:

1.  $\mathbf{q}_n$  and  $\mathbf{k}_m$  vectors rotated prior to attention weight computation to reflect relative position  $n - m$
2. Multiple sets of attention weights computed in parallel
3. Feedforward network applies a non-linear transformation to  $\rho_n^{(i+1)}$
4. Resulting outputs form inputs into next layer ( $L$  in total)

$\rho_n^{(L)}$  is input into final prediction layer.

# Loss Function for Word Prediction

Let  $p_{\theta}(\tilde{w}_n = v \mid \mathbf{w}_{<n})$  be the conditional probability that the word at position  $n$  equals  $v$  given preceding words  $\mathbf{w}_{<n}$ .

$\theta$  stacks all neural network parameters (initial embeddings, attention weights, etc.)

The contribution to the loss function of the prediction problem is

$$-\log p_{\theta}(\tilde{w}_n = w_n \mid \mathbf{w}_{<n})$$

Total loss sums over all word prediction problems, estimates  $\theta$  via gradient descent.

# Outline

Basic Concepts

Transformer Models

Using Generative Model for Measurement

Bidirectional Models

# Zero-Shot Learning

A common approach in economics is to use a fitted autoregressive model for **zero-shot learning**.

User supplies a query that combines a document and classification task, uses LLM response as the predicted value, for example

*“Consider the sentence ‘the economy is booming’. Is the sentiment in this sentence positive, neutral, or negative?”*

Called zero-shot learning when no training examples provided; **few-shot learning** adds some correctly classified examples to the query.

Already by GPT-2, and certainly by GPT-3, emergent behavior observed from training on next-word prediction loss.

See [Bybee, 2023] and [Hansen and Kazinnik, 2024] for early applications in economics.

# Algorithmic Description

User supplies query, which is tokenized as  $\mathbf{w} = (w_1, \dots, w_N)$ .

Generation proceeds iteratively:

1. Draw  $w_{N+1} \sim p_{\theta}(w_{N+1} \mid \mathbf{w})$
2. Append drawn token:  $\mathbf{w} \leftarrow (\mathbf{w}, w_{N+1})$
3. Draw  $w_{N+2} \sim p_{\theta}(w_{N+2} \mid \mathbf{w})$
4. Append, repeat until end-of-sequence token drawn

Result is generated sequence  $(w_{N+1}, w_{N+2}, \dots, w_{N+T})$  where  $w_{N+T}$  is end-of-sequence token.

Final answer is parsed from generated sequence to recover predicted class (e.g. “positive”, “neutral”, “negative”).

# Sampling Details

Let  $p_v \equiv p_\theta(\tilde{w}_n = v \mid \mathbf{w}_{<n})$ .

Several strategies for sampling:

1. **Greedy**: deterministically select  $\arg \max_v p_v$
2. **Temperature**  $\tau$ : sample using weights  $p_v(\tau) \propto p_v^{1/\tau}$ .  
 $\tau \rightarrow 0$  recovers greedy;  $\tau = 1$  uses raw distribution.
3. **Top- $k$** : restrict sampling to  $k$  tokens with highest  $p_v$ , renormalize
4. **Top- $p$  (nucleus)**: restrict to smallest set with cumulative probability  $\geq p$ , renormalize

For classification tasks,  $\tau = 0$  (greedy) standard for reproducibility.

For open-ended generation,  $\tau \in [0.7, 1.0]$  with top- $p \approx 0.9$  common.

# Outline

Basic Concepts

Transformer Models

Using Generative Model for Measurement

**Bidirectional Models**

# Bidirectional Models and Document Encoding

A bidirectional model predicts words using both preceding *and* subsequent tokens.

Neural network trained to produce distribution  $p_{\theta}(\tilde{w}_n = v \mid \mathbf{w}_{-n})$ .

Main architectural difference is that attention weights computed over all context words surrounding attended words.

Bidirectional models can't generate language, but they nevertheless have value:

1. Embedding/encoding documents, e.g. for similarity comparison
2. Foundation model for predicting numeric metadata

Because bidirectional models use full structure of document, their embeddings are potentially richer than those from autoregressive models.

# BERT

Important example is BERT (Bidirectional Encoding Representations from Transformers) [Devlin et al., 2019].

Trained on BooksCorpus (800M words) and English Wikipedia (2,500M words).

**Masked language modeling.** 15% of words randomly masked and given [MASK] token. [MASK] token embeddings built to successfully predict underlying word.

Original paper had next-sentence prediction but has since been dropped from loss function in extensions [Liu et al., 2019].

Base model has twelve layers, 768-dimensional embeddings, 110M parameters.

# References I

Bybee, J. L. (2023).

The Ghost in the Machine: Generating Beliefs with Large Language Models.  
Working Paper.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.  
In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Hansen, A. L. and Kazinnik, S. (2024).

Can ChatGPT Decipher Fedspeak?

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019).

RoBERTa: A Robustly Optimized BERT Pretraining Approach.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013).

Distributed Representations of Words and Phrases and their Compositionality.  
[arXiv:1310.4546 \[cs, stat\]](https://arxiv.org/abs/1310.4546).

# References II

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017).

Attention is All you Need.

In Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc.